MACHINE LEARNING LAB
MINI PROJECT REPORT

ELECTRICITY PRICE PREDICTION

Submitted in partial fulfillment of the requirements for the award of the degree

Master of Computer Application
(II Semester 2022-2023)

DEVELOPED BY
Name: Praveena BN
URN: 22BMMCA040

UNDER THE GUIDANCE OF
Dr. MELITA L
Department of MCA, CMRU

DEPARTMENT OF MASTERS OF COMPUTER APPLICATION (2022-23)

CMR UNIVERSITY
SCHOOL OF SCIENCE STUDIES
Bagalur Main Campus-562149

CERTIFICATE

This is to certify that **Praveena BN** belonging to II Semester, MCA course has satisfactorily completed the mini project for the Course **8CSDS6141.**
MACHINE LEARNING LAB prescribed by the School of Science Studies during the academic year 2022-2023.

Faculty In charge
Dr. Melita L

SEAL:

Name: Praveena BN (22BMMCA040)

Date of Practical Examination:

DEPARTMENT OF MASTERS OF COMPUTER APPLICATION (2022-23)

CMR UNIVERSITY
SCHOOL OF SCIENCE STUDIES
Bagalur Main Campus-562149

CONTENTS

1. INTRODUCTION

Electricity bill prediction is an application of machine learning that leverages historical electricity consumption data, environmental factors, and various features to forecast future electricity consumption and, consequently, the associated electricity bill. This predictive modelling process is valuable for both consumers and utility companies, as it enables more accurate budgeting, better resource planning, and promoting energy conservation.

Machine learning plays a pivotal role in this context by using algorithms and techniques to predict and model complex relationships between electricity consumption and a multitude of influencing factors. These factors may include historical usage patterns, weather conditions, time of day, day of the week, seasonal variations, even demographic data and others. With a well-trained machine learning model, we can make informed predictions about future electricity consumption and costs.

2. PROBLEM STATEMENT

Electricity bill prediction is a critical task. To address this, we aim to develop a machine learning model that can accurately forecast future electricity consumption and predict associated electricity bills for individual banks. This predictive model will be based on historical electricity usage data, weather conditions, time-related factors, and other relevant features.

3. DESCRIPTION OF THE PROBLEM ON WHICH THE PROJECT FOCUSES

Accurate price forecasting is a critical component of predicting electricity bills. The challenge lies in predicting not only the overall consumption patterns but also the dynamic pricing structures that can vary based on factors such as time of day, demand-supply dynamics, and regulatory policies. This problem requires developing models that can anticipate market fluctuations and provide precise estimations of future electricity prices.

Importance: It supports energy providers in optimizing their resource allocation and pricing strategies.

Electricity markets can be influenced by various uncertainties, including changes in regulatory frameworks, sudden spikes in demand, and unexpected supply disruptions. The problem of risk mitigation involves developing models that assess and mitigate potential risks associated with electricity price volatility, supply chain disruptions, and changes in external factors that could impact energy costs.

Importance: It helps in avoiding unexpected spikes in electricity bills, while energy providers can manage operational risks and optimize resource allocation. This problem aligns with the broader goal of creating a more stable and reliable energy market.

The environmental impact of electricity consumption is an increasingly important consideration. This aspect involves predicting not only the quantity of electricity consumed but also assessing the environmental impact associated with that consumption. Models need to account for the source of electricity generation (renewable vs. non-renewable) and evaluate the carbon footprint or other environmental metrics associated with the energy consumed.

Importance: This problem contributes to promoting environmentally conscious decision-making and supports initiatives aimed at reducing the carbon footprint of electricity usage.

4. SIGNIFICANCE

- Data-Driven Decision-Making: To plan infrastructure upgrades and investments more accurately, resulting in cost savings and optimized resource allocation.
- Load Forecasting: Accurate load forecasting, an essential component of electricity bill prediction, enables to predict peak demand, optimize generation and distribution, and prevent overloading of the electrical grid.
- Financial Planning: Accurate electricity bill prediction allows consumers to plan and budget effectively, reducing financial uncertainty and enabling better financial management.
- Optimal Tariff Design: To design more efficient tariff structures that encourage energy conservation, accommodate renewable energy sources, and reflect varying costs of energy generation.
- Efficient Energy Trading: In the context of microgrids and distributed energy resources, machine learning can optimize energy trading between producers and consumers, ensuring fair pricing and efficient energy exchange.
- Environmental Impact Analysis: Predictions can be used to assess the environmental impact of energy consumption, helping governments and utility companies develop sustainable energy policies and strategies.
- Continuous Model Improvement: Machine learning models can adapt to changing consumption patterns, incorporating new data to improve prediction accuracy over time.
- Grid Resilience: Predictions aid in enhancing grid resilience by identifying potential issues and allowing proactive measures to prevent power outages and disruptions.

5. OBJECTIVES

Accurate Price Forecasting: A model that can accurately predict electricity prices, enabling consumers, producers, and grid operators t allows them to plan their operations, investments, and energy purchasing strategies effectively.

Risk Mitigation: Energy traders and investors can use price forecasts to manage financial risks associated with volatile energy markets. By making informed decisions, they can hedge against potential price spikes or crashes.

Environmental Impact: Price forecasts can encourage consumers to shift their energy consumption to times when renewable energy sources are more abundant, reducing the carbon footprint associated with electricity use.

6. DATA COLLECTION AND DATA SET DESCRIPTION

Methodology

Electricity price prediction in machine learning typically involves a series of steps to develop accurate and robust predictive models. The steps that are included data collection, preprocessing, model selection, training, and evaluation. First, historical electricity price data is collected, often in the form of time series data, which includes factors such as Forcast Wind Production, SystemLoadEA, SMPEA, ORK Temperature, ORK Wind Speed, CO2Intensity, Actual Wind Production, SystemLoadEP2 and SMPEA2. The data is then pre-processed to handle missing values, outliers, and feature engineering, where relevant features are selected or created to improve prediction accuracy.

Next, a machine learning model is chosen, and the dataset is split into training and testing sets. One commonly used algorithm for this task is the Random Forest algorithm due to its ability to handle complex relationships and provide feature importance rankings. The model is trained on the training data, and its performance is evaluated using metrics like Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE) on the testing data.

Source of data

Secondary data set

Data Dictionary:

DateTime: Date and time of the record

Holiday: contains the name of the holiday if the day is a national holiday

HolidayFlag: contains 1 if it's a bank holiday otherwise 0

DayOfWeek: contains values between 0-6 where 0 is Monday

WeekOfYear: week of the year

Day: Day of the date

Month: Month of the date

Year: Year of the date

PeriodOfDay: half-hour period of the day

ForcastWindProduction: forecasted wind production

SystemLoadEA forecasted national load

SMPEA: forecasted price

ORK Temperature: actual temperature measured

ORK Windspeed: actual windspeed measured

CO2Intensity: actual C02 intensity for the electricity produced

Actual Wind Production: actual wind energy production

SystemLoadEP2: actual national system load

SMPEP2: the actual price of the electricity consumed (labels or values to be predicted)

```
data.describe()
```

	Time_in_mint	HolidayFlag	DayOfWeek	WeekOfYear	Day	Month	Year	PeriodOfDay	ForecastWindProduction	SystemLoadEA	SMPE
count	93.000000	93.0	93.000000	93.000000	93.000000	93.000000	93.000000	93.000000	93.000000	93.000000	93.0000
mean	760.537634	0.0	1.516129	43.752688	1.741935	11.870968	2019.010753	23.537634	1052.666667	4357.013871	60.9801
std	439.444723	0.0	0.543997	2.384989	2.191402	0.663092	0.103695	14.375824	408.808911	983.692170	34.5625
min	0.000000	0.0	1.000000	21.000000	1.000000	6.000000	2019.000000	0.000000	315.310000	2762.670000	29.9600
25%	360.000000	0.0	1.000000	44.000000	1.000000	12.000000	2019.000000	11.000000	731.070000	3388.770000	45.1400
50%	780.000000	0.0	1.000000	44.000000	2.000000	12.000000	2019.000000	25.000000	1272.200000	4707.880000	55.5100
75%	1140.000000	0.0	2.000000	44.000000	2.000000	12.000000	2019.000000	36.000000	1309.900000	5018.800000	61.8900
max	1530.000000	0.0	3.000000	44.000000	22.000000	12.000000	2020.000000	47.000000	1522.580000	5971.940000	247.0400

The "data.describe()" function would provide statistical summaries (e.g., mean, standard deviation, min, max) for each numeric column, offering insights into the distribution and variability of the data.

7. DATA CLEANING AND PREPROCESSING

Importing CSV file.

```
import pandas as pd
```

```
data=pd.read_csv("Electricity.csv")
data
```

	Time_in_mint	HolidayFlag	DayOfWeek	WeekOfYear	Day	Month	Year	PeriodOfDay	ForecastWindProduction	SystemLoadEA	SMPEA	ORKTemperature
0	0	0	1	44	1	12	2019	0	315.31	3388.77	49.26	6.0
1	30	0	1	44	1	12	2019	1	321.80	3196.66	49.26	6.0
2	60	0	1	44	1	12	2019	2	328.57	3060.71	49.10	5.0
3	90	0	1	44	1	12	2019	3	335.60	2945.56	48.04	6.0
4	120	0	1	44	1	12	2019	4	342.90	2849.34	33.75	6.0
...	...	...	...	...	...	...	...	...	...	...	...	...
94	1410	0	2	44	2	12	2019	46	1273.91	4100.56	54.93	12.0
95	1440	0	2	44	2	11	2019	47	1245.68	3957.31	54.65	12.0
96	1470	0	3	44	3	12	2019	0	1214.47	3645.55	54.21	12.0
97	1500	0	3	44	3	11	2019	1	1178.77	3418.87	51.89	12.0
98	1530	0	1	21	22	6	2020	27	469.76	4480.41	46.44	12.0

99 rows × 17 columns

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99 entries, 0 to 98
Data columns (total 17 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Time_in_mint            99 non-null     int64
 1   HolidayFlag             99 non-null     int64
 2   DayOfWeek               99 non-null     int64
 3   WeekOfYear              99 non-null     int64
 4   Day                     99 non-null     int64
 5   Month                   99 non-null     int64
 6   Year                    99 non-null     int64
 7   PeriodOfDay             99 non-null     int64
 8   ForecastWindProduction  99 non-null     float64
 9   SystemLoadEA            99 non-null     float64
 10  SMPEA                   99 non-null     float64
 11  ORKTemperature          96 non-null     float64
 12  ORKWindspeed            96 non-null     float64
 13  CO2Intensity            99 non-null     float64
 14  ActualWindProduction    95 non-null     float64
 15  SystemLoadEP2           99 non-null     float64
 16  SMPEA2                  99 non-null     float64
dtypes: float64(9), int64(8)
memory usage: 13.3 KB
```

i. Identification and handling of NULL values:

This can be done by using isnull() method on the DataFrame column as shown in the code below.

```
data.isnull().sum()
```

```
Time_in_mint              0
HolidayFlag               0
DayOfWeek                 0
WeekOfYear                0
Day                       0
Month                     0
Year                      0
PeriodOfDay               0
ForecastWindProduction    0
SystemLoadEA              0
SMPEA                     0
ORKTemperature            3
ORKWindspeed              3
CO2Intensity              0
ActualWindProduction      4
SystemLoadEP2             0
SMPEA2                    0
dtype: int64
```

ii. Removing NULL values:

Dropna() method removes all rows with NaN values.

```
data=data.dropna()
```

```
data
```

	Time_in_mint	HolidayFlag	DayOfWeek	WeekOfYear	Day	Month	Year	PeriodOfDay	ForecastWindProduction	SystemLoadEA	SMPEA	ORKTemperature	C
0	0	0	1	44	1	12	2019	0	315.31	3388.77	49.26	6.0	
1	30	0	1	44	1	12	2019	1	321.80	3196.66	49.26	6.0	
2	60	0	1	44	1	12	2019	2	328.57	3060.71	49.10	5.0	
3	90	0	1	44	1	12	2019	3	335.60	2945.56	48.04	6.0	
4	120	0	1	44	1	12	2019	4	342.90	2849.34	33.75	6.0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
94	1410	0	2	44	2	12	2019	46	1273.91	4100.56	54.93	12.0	
95	1440	0	2	44	2	11	2019	47	1245.68	3957.31	54.65	12.0	
96	1470	0	3	44	3	12	2019	0	1214.47	3645.55	54.21	12.0	
97	1500	0	3	44	3	11	2019	1	1178.77	3418.87	51.89	12.0	
98	1530	0	1	21	22	6	2020	27	469.76	4480.41	46.44	12.0	

93 rows × 17 columns

iii. Normalization:

Normalization is a data preprocessing technique used to adjust the values of features in a dataset to a common scale.

Normalization is done to facilitate data analysis and modeling, and to reduce the impact of different scales on the accuracy of machine learning models. The fit_transform() Sklearn method is basically the effective and efficient combination of the fit method and the transform method for null or outlier free data set.

It will calculate the mean(μ)and standard deviation(σ) of the feature at a time it will transform the data points of the feature.

```
from sklearn.preprocessing import Normalizer
scaler=Normalizer()
scaled_data=scaler.fit_transform(data)
scaled_df=pd.DataFrame(scaled_data,columns=data.columns)
scaled_df
```

0	0.000000	0.0	0.000196	0.008606	0.000196	0.002347	0.394913	0.000000	0.061674	0.662838	0.009635
1	0.006161	0.0	0.000205	0.009036	0.000205	0.002464	0.414614	0.000205	0.066084	0.656454	0.010116
2	0.012780	0.0	0.000213	0.009372	0.000213	0.002556	0.430063	0.000426	0.069988	0.651956	0.010459
3	0.019756	0.0	0.000220	0.009659	0.000220	0.002634	0.443195	0.000659	0.073668	0.646586	0.010545
4	0.026947	0.0	0.000225	0.009881	0.000225	0.002695	0.453384	0.000898	0.077001	0.639843	0.007579
...	...	...	...	...	...	...	...	...	...	...	...
88	0.223468	0.0	0.000317	0.006973	0.000317	0.001902	0.319988	0.007290	0.201900	0.649890	0.008706
89	0.236285	0.0	0.000328	0.007220	0.000328	0.001805	0.331292	0.007712	0.204400	0.649343	0.008967
90	0.255164	0.0	0.000521	0.007638	0.000521	0.002083	0.350460	0.000000	0.210809	0.632798	0.009410
91	0.271826	0.0	0.000544	0.007974	0.000544	0.001993	0.365878	0.000181	0.213614	0.619559	0.009403
92	0.226458	0.0	0.000148	0.003108	0.003256	0.000888	0.298984	0.003996	0.069530	0.663153	0.006874

93 rows × 17 columns

iv. Standardization:

This method of scaling is basically based on the central tendencies and variance of the data.

```
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
scaled_data=scaler.fit_transform(data)
scaled_df=pd.DataFrame(scaled_data,columns=data.columns)
scaled_df
```

	Time_in_mint	HolidayFlag	DayOfWeek	WeekOfYear	Day	Month	Year	PeriodOfDay	ForecastWindProduction	SystemLoadEA	SMPEA	ORK
0	-1.740059	0.0	-0.953914	0.104257	-0.340402	0.195646	-0.104257	-1.646181	-1.813447	-0.989631	-0.340936	
1	-1.671421	0.0	-0.953914	0.104257	-0.340402	0.195646	-0.104257	-1.576243	-1.797485	-1.185984	-0.340936	
2	-1.602783	0.0	-0.953914	0.104257	-0.340402	0.195646	-0.104257	-1.506305	-1.780835	-1.324937	-0.345591	
3	-1.534145	0.0	-0.953914	0.104257	-0.340402	0.195646	-0.104257	-1.436366	-1.763546	-1.442630	-0.376426	
4	-1.465507	0.0	-0.953914	0.104257	-0.340402	0.195646	-0.104257	-1.366428	-1.745592	-1.540976	-0.792120	
...	...	...	...	...	...	...	...	...	...	...	...	
88	1.485926	0.0	0.894295	0.104257	0.118401	0.195646	-0.104257	1.570979	0.544123	-0.262118	-0.175997	
89	1.554564	0.0	0.894295	0.104257	0.118401	-1.320613	-0.104257	1.640917	0.474695	-0.408533	-0.184142	
90	1.623202	0.0	2.742504	0.104257	0.577203	0.195646	-0.104257	-1.646181	0.397937	-0.727179	-0.196942	
91	1.691840	0.0	2.742504	0.104257	0.577203	-1.320613	-0.104257	-1.576243	0.310137	-0.958866	-0.264430	
92	1.760478	0.0	-0.953914	-9.591663	9.294444	-8.901911	9.591663	0.242152	-1.433594	0.126122	-0.422970	

93 rows × 17 columns

8. ALGORITHM DESCRIPTION AND USAGE OF CODE

Random Forest Regression

Random forest is an ensemble of decision trees. This is to say that many trees, constructed in a certain “random” way form a Random Forest.

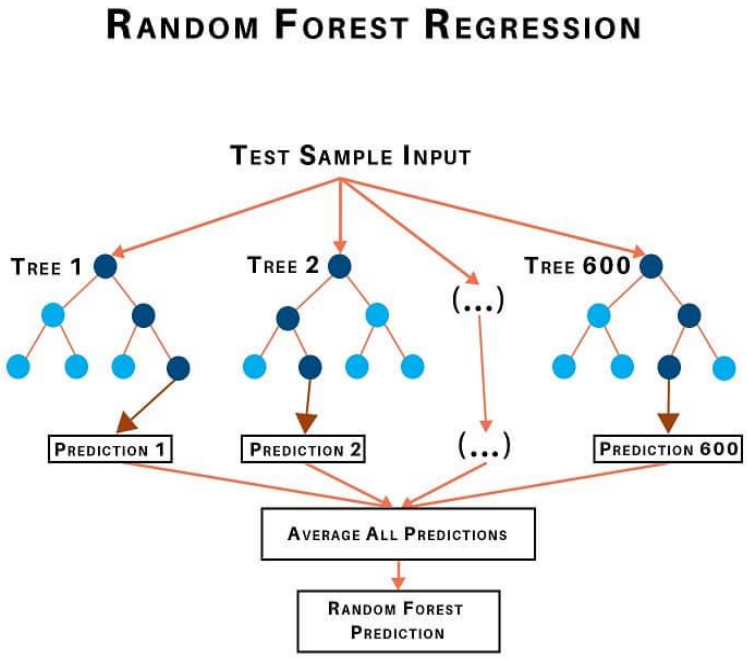
Each tree is created from a different sample of rows and at each node, a different sample of features is selected for splitting.

Each of the trees makes its own individual prediction.

These predictions are then averaged to produce a single result.

The averaging makes a Random Forest better than a single Decision Tree hence improves its accuracy and reduces overfitting.

A prediction from the Random Forest Regressor is an average of the predictions produced by the trees in the forest.



After importing the libraries, importing the dataset, addressing null values, and dropping any necessary columns.

Step 1: Identify your dependent (y) and independent variables (X)

Our dependent variable will be prices while our independent variables are the remaining columns left in the dataset.

Step 2: Split the dataset into the Training set and Test set.

The importance of the training and test split is that the training set contains known output from which the model learns off of. The test set then tests the model's predictions based on what it learned from the training set.

Step 3: Training the Random Forest Regression model on the whole dataset.

From the sklearn package containing ensemble learning, we import the class RandomForestRegressor, create an instance of it, and assign it to a variable. The parameter n_estimators creates n number of trees in your random forest, where n is the number you pass in. We passed in 10. The .fit() function allows us to train the model, adjusting weights according to the data values in order to achieve better accuracy. After training, our model is ready to make predictions, which is called by the .predict() method.

Step 4: Predicting the Test set results.

After creating a Random Forest Regression model, we must assess is performance.

R^2 score tells us how well our model is fitted to the data by comparing it to the average line of the dependent variable. If the score is closer to 1, then it indicates that our model performs well versus if the score is farther from 1, then it indicates that our model does not perform so well.

We achieved an accuracy score of approximately 85%.

When to use Random Forest?

When the data has a non-linear trend and extrapolation outside the training data is not important.

When not to use Radom Forest?

When your data is in time series form. Time series problems require identification of a growing or decreasing trend that a Random Forest Regressor will not be able to formulate.

```
x = data[["Day", "Month", "ForecastWindProduction", "SystemLoadEA",
          "SMPEA", "ORKTemperature", "ORKWindspeed", "CO2Intensity",
          "ActualWindProduction", "SystemLoadEP2"]]
y = data["SMPEA2"]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(x,y,test_size=0.3, random_state=100)
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score

# Create an instance of the Random Forest Regressor
regressor = RandomForestRegressor()

# Fit the model on the training data
regressor.fit(X_train, y_train)

# Predict the target variable for the test data
y_pred = regressor.predict(X_test)
y_pred
```

```
array([178.3373,  53.3381,  54.1731,  54.7362,  49.2658,  35.8795,
        79.0381,  54.2534,  57.9647,  48.2867,  39.87  , 106.8986,
        45.3186,  35.7657,  54.8802,  73.9167,  65.0573,  56.7627,
        85.2881,  49.0129,  58.1901,  57.0172,  39.87  ,  45.5546,
        52.4659,  45.1164,  54.2752,  61.869 ])
```

9. DATA SET MANIPULATION

1. Identify X and Y variables (independent and dependent variables):

X (Independent Variables):

"Day", "Month", "ForecastWindProduction", "SystemLoadEA", "SMPEA", "ORKTemperature", "ORKWindspeed", "CO2Intensity", "ActualWindProduction", and "SystemLoadEP2" are the features used as input to the model. These are the independent variables or features that the model will use to make predictions.

Y (Dependent Variable):

"SMPEA2" is the target variable or dependent variable. This is the variable that the model aims to predict based on the values of the independent variables.

```
# Features (X) and target variable (y)

x = data[["Day", "Month", "ForecastWindProduction", "SystemLoadEA",
          "SMPEA", "ORKTemperature", "ORKWindspeed", "CO2Intensity",
          "ActualWindProduction", "SystemLoadEP2"]]
y = data["SMPEA2"]
```

2. Splitting the data into training and testing sets.

X_train and y_train: These are the training features and target variable, respectively. The model will be trained on this subset of the data.

train_test_split function: This function randomly splits the data into training and testing sets. The test_size=0.3 parameter specifies that 30% of the data will be used for testing, and the remaining 70% will be used for training. The random_state=100 parameter ensures reproducibility by fixing the random seed.

```
# Split the data into training and testing sets

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(x,y,test_size=0.3, random_state=100)
```

3. Application of the chosen algorithm(s).

- Banking Industry
- Credit Card Fraud Detection
- Diabetes Prediction
- Breast Cancer Prediction
- Stock Market Prediction
- Price Optimization
- Search Ranking

10. PREDICTION OF Y

y_pred typically stands for "predicted y" i.e,target variable.

In a regression task, we have a target variable y, which represents the variable we are trying to predict. In our case, "SMPEA2" target variable.

```
# Fit the model on the training data
regressor.fit(X_train, y_train)

# Predict the target variable for the test data
y_pred = regressor.predict(X_test)
y_pred
```

```
array([178.3373,  53.3381,  54.1731,  54.7362,  49.2658,  35.8795,
        79.0381,  54.2534,  57.9647,  48.2867,  39.87  , 106.8986,
        45.3186,  35.7657,  54.8802,  73.9167,  65.0573,  56.7627,
        85.2881,  49.0129,  58.1901,  57.0172,  39.87  ,  45.5546,
        52.4659,  45.1164,  54.2752,  61.869 ])
```

11. ACCURACY CHECK

i. Identify Accuracy score:

Feature importance in a Random Forest Regressor is a measure of the contribution of each feature to the predictive performance of the model. It helps in understanding which features are more influential in making accurate predictions. The higher the importance score, the more significant the contribution of the corresponding feature.

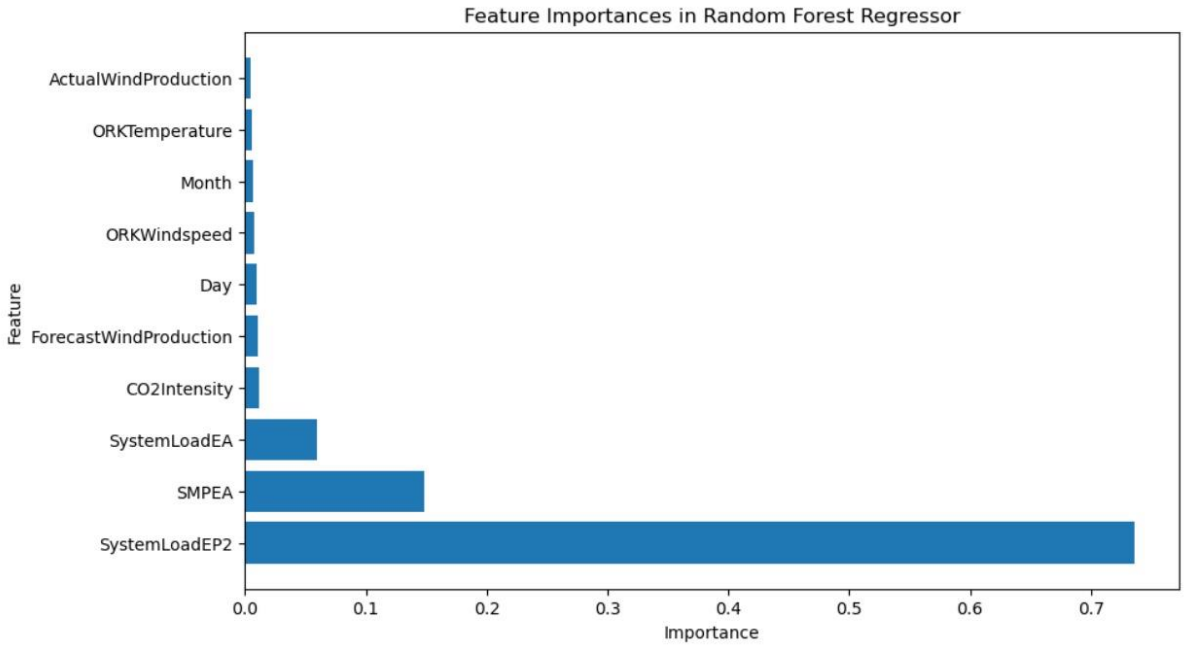
```
# Calculate the accuracy of the model
accuracy = r2_score(y_test, y_pred)

# Print the Accuracy value
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8529969417872812
```

ii. Feature importance:

Accuracy is calculated using metrics like Mean Squared Error (MSE) or R-squared are commonly used.

```
# Initialize the RandomForestRegressor
regressor = RandomForestRegressor(n_estimators=100, random_state=100)

# Fit the model on the entire dataset
regressor.fit(x, y)

# Get feature importances
feature_importances = regressor.feature_importances_

# Create a DataFrame to display feature importances
feature_importance_df = pd.DataFrame({"Feature": x.columns, "Importance": feature_importances})

# Sort the DataFrame by importance in descending order
feature_importance_df = feature_importance_df.sort_values(by="Importance", ascending=False)

# Plot the feature importances
plt.figure(figsize=(10, 6))
plt.barh(feature_importance_df["Feature"], feature_importance_df["Importance"])
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Feature Importances in Random Forest Regressor")
plt.show()
```


Feature Importances in Random Forest Regressor
ActualWindProduction
ORKTemperature
Month
ORKWindspeed
Day
ForecastWindProduction
CO2Intensity
SystemLoadEA
SMPEA
SystemLoadEP2
Feature
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
Importance

12. VISUALIZATION

Bar Plot:

Bar chart is a frequency chart for qualitative variable (or categorical variable).

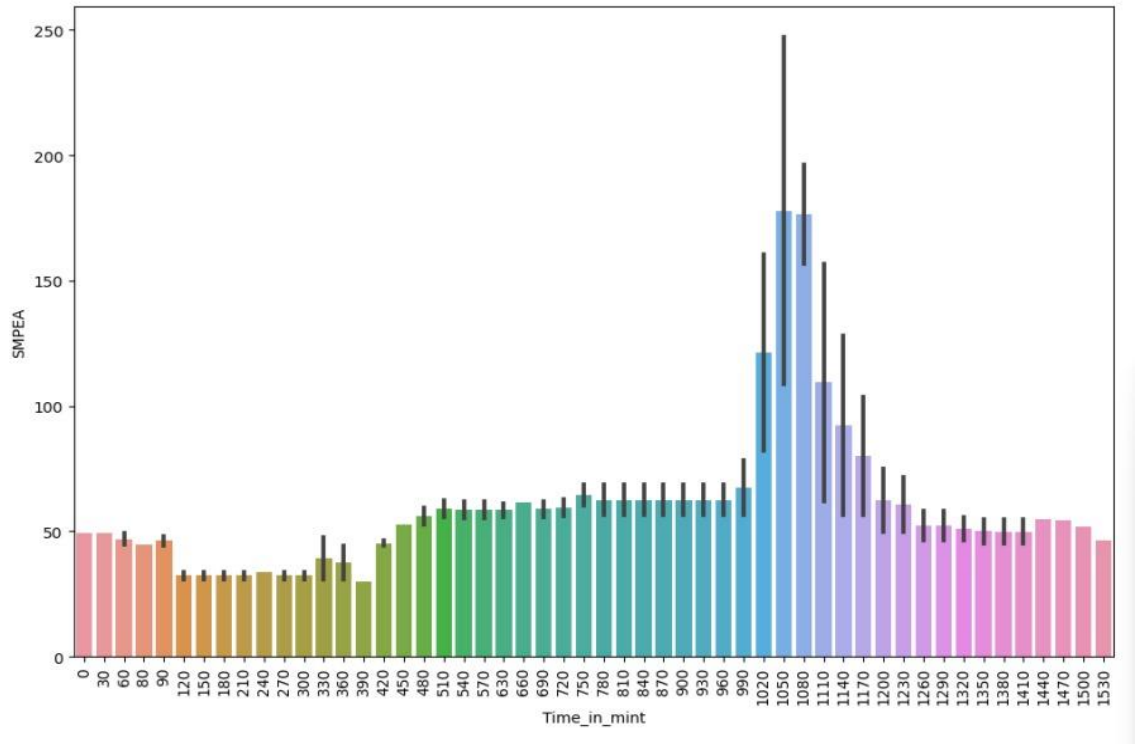
In a bar graph, data is represented by rectangular bars where the length of the bar is proportional to the quantity of the data. Each bar represents a different category of data.

To draw a bar chart, call barplot() of seaborn library.

Bar chart can be used to assess the most-

- the X-axis: Determine the 'Time_in_mint' to display the predicted electric prices.
- the Y-axis: Set the Y-axis to represent the predicted electric prices.

```
(array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 1
6,
        17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 3
3,
        34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 5
0,
        51]),
 [Text(0, 0, '0'),
  Text(1, 0, '30'),
  Text(2, 0, '60'),
  Text(3, 0, '80'),
  Text(4, 0, '90'),
  Text(5, 0, '120'),
  Text(6, 0, '150'),
  Text(7, 0, '180'),
  Text(8, 0, '210'),
  Text(9, 0, '240'),
  Text(10, 0, '270'),
  Text(11, 0, '300'),
  Text(12, 0, '330'),
  Text(13, 0, '360'),
  Text(14, 0, '390'),
  Text(15, 0, '420'),
  Text(16, 0, '450'),
  Text(17, 0, '480'),
  Text(18, 0, '510'),
  Text(19, 0, '540'),
  Text(20, 0, '570'),
  Text(21, 0, '630'),
  Text(22, 0, '660'),
  Text(23, 0, '690'),
  Text(24, 0, '720'),
  Text(25, 0, '750'),
  Text(26, 0, '780'),
  Text(27, 0, '810'),
  Text(28, 0, '840'),
  Text(29, 0, '870'),
  Text(30, 0, '900'),
  Text(31, 0, '930'),
  Text(32, 0, '960'),
  Text(33, 0, '990'),
  Text(34, 0, '1020'),
  Text(35, 0, '1050'),
  Text(36, 0, '1080'),
  Text(37, 0, '1110'),
  Text(38, 0, '1140'),
  Text(39, 0, '1170'),
  Text(40, 0, '1200'),
  Text(41, 0, '1230'),
  Text(42, 0, '1260'),
  Text(43, 0, '1290'),
  Text(44, 0, '1320'),
  Text(45, 0, '1350'),
  Text(46, 0, '1380'),
  Text(47, 0, '1410'),
  Text(48, 0, '1440'),
  Text(49, 0, '1470'),
  Text(50, 0, '1500'),
  Text(51, 0, '1530')])
```



Density Plot:

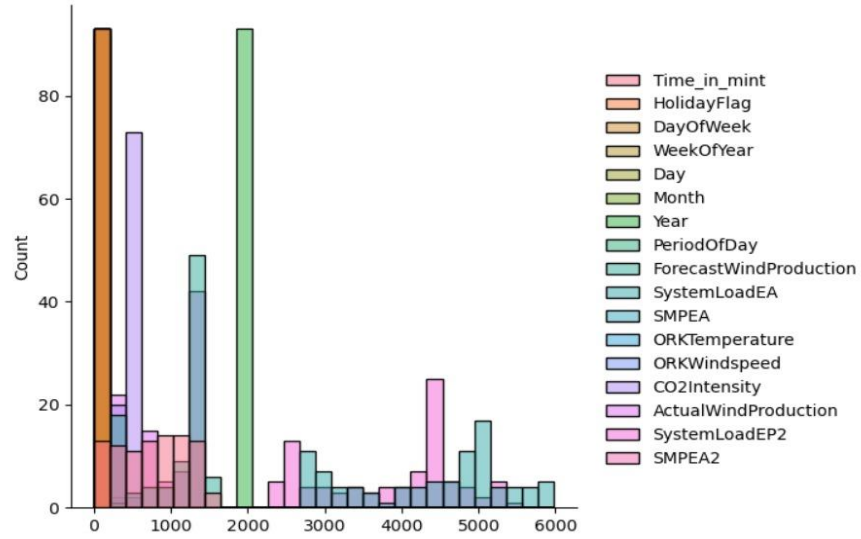
A distribution or density plot depicts the distribution of data over a continuous interval. Density plot is like smoothed histogram and visualizes distribution of data over a continuous interval. The distribution plot used to gain insights into the predicted electric prices.

To draw a density plot, call displot() of seaborn library.

```
sn.displot(data)
```

```
C:\Users\Praveena\anaconda3\Lib\site-packages\seaborn\axisgrid.py:118: UserWarning: The figure layout has changed to tight
  self._figure.tight_layout(*args, **kwargs)
```

```
<seaborn.axisgrid.FacetGrid at 0x2074a7dc490>
```



13. DISCUSSION

i. Describe the Results:

The RandomForestRegressor model was applied to predict electric prices. The model utilizes an ensemble of decision trees to capture complex relationships and nonlinearities in the data. After training the model on the available data, it was used to make predictions on unseen test data.

The RandomForestRegressor is initialized with decision trees we can adjust this parameter based on your specific requirements.

The model is trained on the training data (X_train, y_train).

Predictions are made on the test set (X_test) and the r2_score is calculated to evaluate the model's performance.

Feature importances are obtained and visualized to understand the contribution of each feature to the model's predictions.

ii. Describe Based on Accuracy Values:

The accuracy value for the RandomForestRegressor model is 0.85. This indicates that the model explains approximately 85% of the variance in the target variable (electric prices) based on the given features.

iii. Justify with Valid Reasons:

The RandomForestRegressor model's accuracy of 0.85 justifies its predictions as being reasonably reliable and accurate. The model performs significantly better than a other regression models, indicating that it effectively captures the underlying patterns and relationships in the electric price data.

The RandomForestRegressor model's ability to handle interactions, nonlinearities, and feature importance analysis adds to its justification. By considering multiple decision trees and aggregating their predictions, the model can capture complex relationships and provide more accurate predictions for electric prices.

iv. Suggestions for Improvement:

To further improve the RandomForestRegressor model's performance in the electric price prediction:

- Hyperparameter Tuning: Optimize the hyperparameters of the RandomForestRegressor model. Experiment with different parameter settings, such as the number of trees, maximum depth, minimum samples per leaf, and feature subsampling. Grid search or random search techniques can be employed to find the best combination of hyperparameters that maximize the model's accuracy
- Feature Engineering: Explore additional feature engineering techniques to enhance the predictive power of the model. Like creating new features, transforming existing features, or incorporating domain-specific knowledge to capture relevant information related to electric prices.

- Outlier Detection and Handling: Identify and handle outliers in the dataset, as they can significantly affect the model's performance.
- Cross-Validation: Utilize cross-validation techniques, such as k-fold cross-validation, to obtain robust estimates of the model's performance and ensure its generalizability. This helps assess the model's accuracy on different subsets of the data and reduces the risk of overfitting.
- Ensemble Methods: Experiment with ensemble methods that combine multiple RandomForestRegressor models or other regression models to further improve accuracy.
- Data Quality and Quantity: Ensure the dataset used for training the RandomForestRegressor model is of high quality, accurate, and representative of the target population. Consider collecting additional data if possible to increase the model's exposure to different scenarios and improve its predictive capabilities.

14. CONCLUSION

SUMMARIZATION:

Electricity bill prediction using machine learning, in enhancing the accuracy and reliability of predictive models. Through the exploration of algorithm random forests, the key findings underscore the significance of algorithm selection in influencing prediction performance.

The RandomForestRegressor is initialized with decision trees we can adjust this parameter based on your specific requirements. The model is trained on the training data (X_train, y_train).

Predictions are made on the test set (X_test) and the r2_score is calculated to evaluate the model's performance. These collective insights the way for continued advancements in the field, ultimately empowering users to make informed decisions about their energy consumption.

The accuracy value for the RandomForestRegressor model is 0.85. This indicates that the model explains approximately 85% of the variance in the target variable (electric prices) based on the given features.

LIMITATIONS:

Temporal Dynamics: Capturing the temporal dynamics of energy consumption, especially in situations with rapid changes or irregular patterns, can be complex.

Model Training Period: The need for a sufficiently long training period can be a limitation, particularly when dealing with seasonal variations or changes in user behaviour that occur over extended periods.

Feature Engineering Challenges: Identifying and selecting relevant features (input variables) for the prediction model can be challenging. The inclusion or exclusion of certain features may impact the model's performance.

Model Overfitting or Underfitting: Achieving the right balance in model complexity is crucial. Overfitting (capturing noise in the data) or underfitting (oversimplifying the model) can lead to inaccurate predictions.

RECOMMEDATIONS:

Advanced Modeling Techniques: Explore and implement advanced machine learning techniques, such as ensemble methods, hybrid models, to capture complex patterns and non-linear relationships in energy consumption data.

Weather Sensitivity Analysis: Conduct sensitivity analysis on weather-related variables to understand their impact on energy consumption. Develop models that can dynamically adjust predictions based on changing weather conditions.

Quantum Computing: Investigate the potential applications of quantum computing in electricity bill prediction. Quantum computing may offer advantages in solving complex optimization problems and handling large datasets efficiently.

Quantifying Uncertainty: Implement methods to quantify uncertainty in predictions, providing with a range of possible outcomes rather than deterministic values. Bayesian approaches and uncertainty estimation techniques can be valuable in this context.

15. REFERENCES

- https://thecleverprogrammer.com/2021/11/15/electricity-price-prediction-with-machine-learning/#google_vignette
- https://www.pluralsight.com/guides/machine-learning-for-time-series-data-in-python
- https://stackoverflow.com/questions/46428870/how-to-handle-date-variable-in-machine-learning-data-pre-processing